SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyse Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Scenario-Based : 20% **Total Marks: 50**

Annexure A

Code of Conduct:

I am Jeevakrishna. R (192511108) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A1 — Scenario-Based Answers

Cryptography and Network Security (CSA51)

The following section addresses each of the five scenario-based questions from Annexure A1. Each answer covers the scenario context, the key security issues involved, the relevant cryptographic concepts applied, and a clearly justified recommendation, in line with the rubric criteria for this assessment (Understanding of Scenario, Identification of Key Issues, Application of Concepts, Decision Making, and Clarity of Response).

Question 1: Secure Password Storage in Web Applications

Scenario: A web application stores user passwords and needs to protect them against database breaches.

Part A — Why Storing Plaintext Passwords Is Insecure

When a web application stores passwords in plaintext, it means the exact characters a user types during registration are saved directly in the database, without any transformation. This creates a single point of catastrophic failure: if an attacker gains unauthorized access to the database — through SQL injection, an insider threat, a misconfigured backup, or a compromised server — every single user's password is immediately and fully exposed with no additional effort required on the attacker's part.

The risk extends well beyond the breached application itself. Because a large proportion of users reuse the same password (or minor variations of it) across multiple websites and services, a plaintext leak from one relatively low-value application can be used to compromise banking portals, email accounts, and corporate systems through a technique known as credential stuffing. This turns a single breach into a cascading, multi-platform security incident.

Key issues this raises:

- No cryptographic barrier exists between the stored data and the attacker — compromise is instantaneous and total.
- Violates the principle of defense in depth, since a single database compromise defeats the entire authentication system.
- Enables offline credential-stuffing attacks against other services due to password reuse.
- Fails regulatory and compliance requirements such as GDPR, PCI-DSS, and ISO/IEC 27001, which mandate protection of stored credentials.
- Destroys user trust and exposes the organization to legal liability and reputational damage.

Part B — Secure Method for Storing Passwords and Justification

The recommended approach is to never store the password itself, but instead store a salted, computationally expensive one-way hash of the password, generated using a dedicated password-hashing algorithm such as bcrypt, scrypt, or Argon2 (Argon2id is currently considered the strongest general-purpose choice).

How the mechanism works:

1. At registration, a unique, random salt (typically 128 bits or more) is generated for each user.

2. The password is concatenated with the salt and passed through the hashing algorithm a large, configurable number of iterations (a 'work factor'), producing a fixed-length hash.
3. Only the salt and the resulting hash are stored in the database — the original password is discarded and never persisted anywhere.
4. At login, the same process is repeated on the submitted password using the stored salt, and the newly computed hash is compared with the stored hash.

Justification for this choice:

- One-way property: Cryptographic hash functions cannot practically be reversed, so even a full database leak does not directly reveal passwords.
- Unique salting defeats precomputed rainbow-table attacks and ensures that two users with the same password produce completely different stored hashes.
- Deliberately slow, memory-hard algorithms (bcrypt/Argon2) make brute-force and GPU/ASIC-accelerated cracking economically impractical, unlike fast general-purpose hashes such as MD5 or SHA-1, which are unsuitable for this purpose.
- The work factor can be increased over time as hardware improves, keeping the scheme future-proof.

Overall, this approach shifts the security guarantee away from 'hope the database is never breached' toward 'even if the database is breached, the attacker still cannot feasibly recover usable passwords' — which is the correct engineering posture for this scenario.

Question 2: Digital Signature for Document Authentication

Scenario: A company wants to ensure that official documents sent electronically are authentic and unaltered.

Part A — How Digital Signatures Can Be Used in This Scenario

A digital signature uses asymmetric (public-key) cryptography to bind a document to the identity of its sender in a way that can be independently verified by anyone who receives it. The company would apply the following process before sending an official document:

5. The sender first runs the document through a cryptographic hash function (such as SHA-256), producing a fixed-size digest that uniquely represents the document's contents.
6. The sender then encrypts this digest using their own private key — this encrypted digest is the digital signature, and it is attached to (or sent along with) the original document.
7. The recipient independently computes the hash of the received document and decrypts the attached signature using the sender's publicly available public key.
8. If the freshly computed hash matches the decrypted value from the signature, the document is confirmed as both authentic and unmodified; if even a single bit differs, the two hashes will not match, and verification fails.

This process is typically implemented using established digital signature standards such as RSA-based signatures or the Digital Signature Algorithm (DSA/ECDSA), often within a Public Key Infrastructure (PKI) that ties public keys to verified identities through digital certificates issued by a trusted Certificate Authority (CA).

Part B — How This Method Ensures Integrity and Non-Repudiation

Integrity:

Because the signature is mathematically tied to the exact hash of the original content, any alteration to the document — even changing a single character — produces a completely different hash value once recomputed. This mismatch immediately reveals tampering, giving the recipient cryptographic assurance that the document received is identical to the document that was signed.

Non-Repudiation:

Since the signature can only have been created using the sender's private key — which, under correct key-management practice, is known only to the sender — successful verification with the sender's corresponding public key proves that the sender, and no one else, produced that signature. This prevents the sender from later denying having sent or authored the document, which is essential for legally binding official communications, contracts, and audit trails.

Supporting decision points for the company:

- Adopt a recognized PKI and obtain certificates from a trusted CA so recipients can validate the sender's public key with confidence.
- Use a strong hash function (SHA-256 or higher) to avoid collision-based forgery.
- Combine digital signatures with a secure channel (e.g., TLS) for transmission, and consider timestamping services to also prove when the document was signed.

Together, these measures give the company a legally and technically defensible mechanism for authenticating official electronic documents.

Question 3: Preventing Replay Attacks in Online Transactions

Scenario: An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

Part A — How Replay Attacks Occur

A replay attack occurs when an attacker intercepts a legitimate, correctly authenticated message — such as a valid payment authorization request — as it travels across the network, and then retransmits (replays) that exact same message at a later time to trick the receiving system into processing it again.

Critically, the attacker does not need to break any encryption or forge a signature to succeed. Because the original message was legitimately authenticated by the real sender, the payment server has no way of distinguishing the replayed copy from the original unless additional protections are in place. This means a single captured transaction — for example, 'transfer $500 from Account A to Account B' — could be resent multiple times, resulting in duplicate or fraudulent charges.

Key issues that make this attack possible:

- Encryption alone (confidentiality) does not guarantee that a message is fresh or being used only once.
- Static or reusable authentication tokens can be captured and resubmitted without needing to be decrypted.
- Lack of transaction-level uniqueness checks allows the same request to be accepted multiple times by the server.
- Public Wi-Fi or unsecured intermediary networks make interception of transaction packets easier.

Part B — Cryptographic Technique to Prevent Replay Attacks

The standard and most effective defense is to combine timestamps and/or sequence numbers with nonces (numbers used once), all protected under a Message Authentication Code (MAC) or digital signature, so that any replay attempt is cryptographically detectable and rejected.

How this works in practice:

9. Each transaction request includes a unique, randomly generated nonce and a timestamp indicating when it was created.
10. The client computes a MAC (e.g., HMAC-SHA256) or digital signature over the transaction data together with the nonce and timestamp, using a shared secret key or the client's private key.
11. The server checks that the timestamp falls within an acceptable, short validity window (e.g., 60 seconds), and verifies that the specific nonce has not already been recorded as 'used' in its short-term cache.
12. If the same nonce is presented again, or the timestamp is outside the valid window, the server rejects the transaction outright, regardless of whether the MAC/signature is valid.

Justification:

- Nonces guarantee that even an identical message cannot be successfully processed twice, since the server tracks which nonces have already been consumed.
- Timestamps bound the attack window, so an attacker has a very limited time to replay a captured message even if a nonce check were somehow bypassed.

- Because the nonce and timestamp are included inside the MAC/signature calculation, an attacker cannot simply strip or modify them without invalidating the authentication check.
- This approach is widely adopted in real-world protocols such as TLS session handling, OAuth 2.0 token exchanges, and challenge-response authentication in payment gateways (e.g., 3-D Secure).

This combination gives the payment system strong protection: authenticity (the message truly came from the claimed sender), integrity (it wasn't altered), and freshness (it cannot be reused), which together comprehensively address the replay vulnerability.

Question 4: Secure Communication over Public Wi-Fi

Scenario: A user accesses sensitive information using a public Wi-Fi network without encryption.

Part A — Risks Involved in This Scenario

Public Wi-Fi networks, particularly those without WPA2/WPA3 encryption or with a shared open password, allow any device on the same network segment to potentially observe traffic passing between the user's device and the access point. This exposes the user to several concrete threats:

- Eavesdropping / Packet Sniffing: An attacker on the same network can use freely available tools to capture unencrypted traffic and read sensitive data such as login credentials, session cookies, or personal messages in plaintext.
- Man-in-the-Middle (MITM) Attacks: An attacker can position themselves between the user and the legitimate access point (e.g., via ARP spoofing or a rogue/'evil twin' access point), intercepting and potentially altering traffic without either party's knowledge.
- Session Hijacking: Captured session tokens or cookies can allow an attacker to impersonate the user on a website without needing their password.
- DNS Spoofing and Phishing Redirection: Attackers can redirect the user to malicious lookalike websites designed to harvest further credentials.
- Malware Injection: Unencrypted HTTP traffic can be modified in transit to inject malicious scripts or downloads into otherwise legitimate web pages.

Part B — Recommended Secure Communication Method and Justification

The recommended solution is for the user to route all traffic through a Virtual Private Network (VPN) using a strong tunneling protocol (such as OpenVPN or WireGuard), in combination with ensuring that any sensitive site accessed is served over HTTPS (TLS 1.2/1.3).

How a VPN protects the user:

13. The VPN client establishes an encrypted tunnel between the user's device and a trusted VPN server, typically using strong symmetric encryption (e.g., AES-256) negotiated via an asymmetric key exchange (e.g., Diffie–Hellman or ECDHE).
14. All outgoing traffic is encrypted before it ever leaves the device, meaning anyone monitoring the public Wi-Fi network only sees unreadable ciphertext, not the actual content of the communication.
15. The VPN server decrypts the traffic and forwards it to its final destination over a trusted network path, masking the user's original IP address and location from the local network.

Why this is the right choice:

- Confidentiality: Even if an attacker captures every packet on the open Wi-Fi network, the encrypted tunnel renders the data unreadable without the corresponding session keys.
- Integrity: Modern VPN protocols include authentication mechanisms (such as HMAC or AEAD ciphers like AES-GCM) that detect any tampering with packets in transit.

- Defense against MITM: Because the VPN tunnel is established using certificate-based or pre-shared-key authentication with the trusted VPN server, a rogue access point cannot successfully impersonate the legitimate endpoint.
- Layered protection: Using HTTPS on top of the VPN adds an additional layer of end-to-end encryption directly with the destination website, so security is not solely dependent on the VPN provider.

Together, a VPN combined with HTTPS transforms an inherently untrusted public network into a secure channel, ensuring that sensitive information remains protected in confidentiality, integrity, and authenticity even in a hostile network environment.

Question 5: Data Integrity in Cloud Storage

Scenario: An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

Part A — How Hashing Can Be Used to Verify Data Integrity

Cryptographic hashing provides a compact, verifiable 'fingerprint' of data that can be used to detect any unauthorized modification, whether caused by malicious tampering, storage corruption, or transmission errors.

Process:

16. Before uploading a file (or dataset) to the cloud, the organization computes its cryptographic hash using an algorithm such as SHA-256, and securely retains this original hash value locally or in a trusted, independent system.
17. The file is then uploaded and stored on the cloud server.
18. At any later point — periodically, or before critical use of the data — the organization (or an automated integrity-checking service) downloads the file, or requests the cloud provider compute its hash server-side, and recalculates the hash using the same algorithm.
19. The newly computed hash is compared against the originally stored hash value: an exact match confirms the data is bit-for-bit identical to what was originally stored, while any mismatch immediately flags that the data has been altered, corrupted, or tampered with.

Because cryptographic hash functions are designed so that even a single-bit change in the input produces a drastically different output (the avalanche effect), and because it is computationally infeasible to find two different inputs producing the same hash (collision resistance), this technique gives strong, efficient integrity verification without needing to compare the entire files byte-by-byte.

Part B — Additional Mechanism to Enhance Trust and Justification

While hashing alone verifies that data has not changed, it does not by itself prove who created the hash or protect against an attacker who modifies both the file and the stored hash together. To close this gap, the organization should additionally implement digital signatures (or a keyed HMAC) over the hash value, ideally supported by a blockchain-based or append-only integrity log for high-value records.

Recommended enhancement:

- Sign the hash: After computing the hash of the file, the organization signs that hash using its private key. This signed hash is stored separately from the cloud provider (e.g., on an internal server or a trusted third-party timestamping/notary service).
- Independent verification path: Because the signature can only be produced with the organization's private key, and verification uses the corresponding public key, an attacker who compromises only the cloud storage cannot forge a matching valid signature for tampered data.
- Timestamping: Adding a trusted timestamp to the signed hash also proves when the data existed in that exact state, which is valuable for compliance, audit trails, and legal evidence.
- Immutable logging: Recording hash values in an append-only or blockchain-based ledger prevents even an insider with cloud administrator access from silently altering the reference hash to match tampered data.

Justification:

This layered approach separates 'detecting change' (hashing) from 'trusting the source and timing of the reference value' (signing and timestamping), ensuring that the organization's trust in its cloud-stored data does not rely solely on the cloud provider's own integrity or on any single unprotected reference value. This significantly strengthens the overall assurance that critical organizational data remains authentic, unaltered, and verifiable at any point in time.

Summary Table: Cryptographic Techniques Applied

The table below consolidates the core cryptographic concept, mechanism, and security property addressed in each scenario, for quick reference.

Scenario	Core Technique	Security Property Achieved
1. Password Storage	Salted hashing (bcrypt / Argon2)	Confidentiality of credentials, resistance to offline cracking
2. Digital Signatures	Asymmetric encryption + hashing (RSA/ECDSA)	Integrity and non-repudiation
3. Replay Attacks	Nonce + timestamp with HMAC/signature	Freshness, authenticity, integrity
4. Public Wi-Fi	VPN tunneling (AES) + HTTPS/TLS	Confidentiality, integrity, MITM resistance
5. Cloud Data Integrity	Hashing (SHA-256) + digital signatures	Data integrity and verifiable trust

Across all five scenarios, a common engineering principle emerges: confidentiality, integrity, authenticity, and non-repudiation are rarely achieved by a single mechanism in isolation. Effective security design combines complementary cryptographic techniques — hashing for integrity, asymmetric cryptography for authenticity and non-repudiation, symmetric encryption for confidentiality at scale, and freshness mechanisms such as nonces and timestamps to prevent reuse-based attacks. This layered approach reflects the defense-in-depth strategy expected in real-world secure system design.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand